DCORTEX

Agentic Crew Ops Advisor

AI-driven operational superintelligence for airline Crew Control

1. Background / Business Context

dCortex builds AI agents for airline operations — a system of action for airlines.

Every airline runs on a plan. Crew are assigned to flights weeks in advance against a forecasted schedule, and on paper the day works perfectly. The day never works perfectly.

The crew called in sick at 5 a.m. Duty-hour and flight-hour clocks run out mid-rotation. Licences and medical certificates lapse. Weather closes a station for six hours. One delay cascades down an aircraft rotation and breaks four downstream flights. Each event can affect several flights and hundreds of passengers — and every fix creates new problems that must be resolved recursively, in real time.

Absorbing all of this is the **Crew Control desk**: a small team who must work out, within minutes, who is affected, which flights are now at risk, which crew can *legally* be moved, who can actually be reached, and what it costs. They do this by cross-referencing rosters, duty clocks, reserve lists and a dense regulatory rulebook across several screens.

The bottleneck is not detecting that something broke. It is **reasoning correctly and fast about the consequences**, from data scattered across many sources. Today that reasoning lives in an experienced controller's head — slow to learn, impossible to scale, and it degrades exactly when it matters most.

This is a universal pattern: logistics, healthcare rostering, field service, energy. The airline setting simply makes it concrete.

2. Problem Definition

Current workflow

When a disruption hits, a controller manually:

1. Identifies the affected crew member and their assignments
2. Traces which flights are now uncovered — including downstream legs in the same pairing
3. Scans the reserve pool for available, qualified, reachable crew
4. Validates each candidate against duty-hour and flight-time limits
5. Weighs cost and knock-on impact
6. Makes the call and notifies the crew

Pain points

- **Fragmented data** — one answer spans rosters, duty clocks, schedules, reserve registers, qualifications and the rulebook
- **Consequence blindness** — the broken flight is obvious; the four that break next are not
- **Legality is exact arithmetic** — an approximate answer is a violation
- **Expertise bottleneck** — only senior controllers can do this fluently, one disruption at a time
- **No reasoning trail** — decisions can't be reviewed, trusted or learned from

The challenge

Build a **Crew Ops Advisor**: a conversational system that understands a controller's question in plain language, reasons over the provided operational dataset, and responds with a clear, correct, explainable answer.

Questions span three tiers. **You are not expected to fully solve all three** — the tiers exist so every team has an on-ramp and no team hits a ceiling.

Tier 1 — Lookup & Retrieval (mandatory)

Answerable directly from the data. No domain modelling required.

- "Who's on reserve at BLR tomorrow?"
- "How many duty hours does C-1042 have left this week?"
- "Which flights depart DEL this afternoon?"
- "List crew whose licence expires in the next 30 days."

Tier 2 — Consequence & Simulation (strongly expected)

Requires reasoning about impact, not just retrieval.

- "Captain C-1042 just called in sick for tomorrow — which flights are now uncrewed?"
- "If I move FO C-2087 onto DX412, does anyone breach a duty limit?"
- "Station BLR is closed 14:00–20:00 — what's the crew impact?"

Tier 3 — Recommendation & Action (stretch)

Requires ranking legal options against real trade-offs.

- "Captain C-1042 is out — what should I do?"
- Expected: ranked, rule-compliant options with cost, legality status, reachability and reasoning.
- Bonus: draft the notification message to the affected crew.

The central engineering challenge

The obvious approach — put the data in the prompt and let the model answer — works for Tier 1 and **fails at Tier 2 and 3**. Legality is exact arithmetic against a rulebook; an LLM that approximates a duty-hour calculation produces answers that are fluent, confident and wrong. Operationally, that is worse than no answer.

So the real question you are answering is architectural:

> **What should the language model do, what should deterministic code do, and how do you compose them into a system that is both conversational and correct?**

Retrieval strategies, tool-calling agents, query generation, a rules engine behind a natural-language front end, hybrid planning — all legitimate. We care far more about how you reason about this trade-off, and how honestly your system handles the limits of its own knowledge, than which framework you pick.

Explainability is mandatory. Every non-trivial answer must carry reasoning a controller can read and challenge. A correct answer with no visible reasoning scores poorly.

3. Tech Stack

Entirely your choice. Guidance, not restriction.

Layer	Options
Backend	Python, Node.js, Java, Go
Frontend	React, Next.js, Streamlit, or a well-built CLI
Database	SQLite (recommended — sufficient), PostgreSQL, MongoDB, DuckDB
AI	Claude, OpenAI, Gemini, LangChain, LlamaIndex, Hugging Face
Cloud	AWS, Azure, GCP, or fully local

The dataset runs on a laptop. Do not spend hackathon hours on infrastructure — a local SQLite file plus a clean interface scores better than an unfinished distributed deployment.

4. Resources Provided

Released in a **starter repository 24 hours before the hackathon**. No setup time should be spent on access.

Synthetic dataset (seeded, identical for all teams)

File	Contents
flights.json	7-day schedule, ~8 stations, ~140 flights, rotations, block times
crew.json	~150 crew — rank, aircraft ratings, base, seniority, reachability
rosters.json	Pairings and duty assignments, consistent with the schedule
duty_clocks.json	Running duty-hour and flight-hour accruals per crew
reserve_pool.json	Reserve crew with on-call windows and standby status
certifications.json	Licence, medical and recurrent-training validity/expiry
rules.json	Machine-readable legality ruleset (below)
costs.json	Callout, overtime, deadhead and penalty rates
risk_signals.json	Pre-computed per-crew disruption-risk scores
scenarios.json	6 worked disruption scenarios with answer keys
questions.json	~40 example questions across Tiers 1–3 with expected answers

Legality ruleset

Simplified, based on publicly published regulatory structures.

Rule ID	Constraint
RULE-FDP-01	Maximum flight duty period of 13 hours, reduced by sectors flown
RULE-DUTY-02	Maximum 60 duty hours in any 7 consecutive days

Rule ID	Constraint
RULE-FLT-03	Maximum 100 flight hours in any 28 consecutive days
RULE-REST-04	Minimum 12 hours rest before commencing duty
RULE-QUAL-05	Crew must hold a valid rating for the assigned aircraft type
RULE-CERT-06	All certifications must be valid on the duty date
RULE-BASE-07	Reserve callout from base only, unless deadhead cost is applied

Sample record shapes

```
// crew.json
{
  "crew_id": "C-1042",
  "name": "A. Nair",
  "rank": "Captain",
  "base": "BLR",
  "ratings": ["A320"],
  "seniority": 14,
  "reachability_minutes": 90
}

// duty_clocks.json
{
  "crew_id": "C-1042",
  "duty_hours_7d": 48.5,
  "flight_hours_28d": 82.0,
  "last_rest_ended": "2026-09-14T22:00:00Z"
}
```

Expected output shape — Tier 2

```
{
  "query": "Captain C-1042 called in sick for 15 Sep",
  "impact": {
    "uncrewed_flights": ["DX412", "DX413", "DX588"],
    "pairing_broken": "P-2291",
    "downstream_risks": [
      { "crew_id": "C-2087", "rule": "RULE-DUTY-02",
        "detail": "Would exceed 60h/7d by 1h20m" }
    ],
    "passengers_affected": 486
  },
  "explanation": "C-1042 operates pairing P-2291 (DX412/413/588 on 15 Sep).
     All three legs are now uncrewed. The nearest substitution, C-2087,
     would breach the 7-day duty limit."
}
```

Expected output shape — Tier 3

```
{
  "options": [
    { "rank": 1, "action": "Assign reserve C-3310",
      "legal": true,
      "rules_checked": ["RULE-FDP-01","RULE-QUAL-05","RULE-BASE-07"],
      "cost_inr": 18500, "coverage": "all 3 flights",
      "reasoning": "BLR-based, A320-rated, on-call 06:00-18:00,
                    reachable in 45 min." },
    { "rank": 2, "action": "Deadhead C-2210 from DEL",
      "legal": true, "cost_inr": 41200, "coverage": "all 3 flights",
      "reasoning": "Legal but incurs deadhead cost and 3h delay to DX412." }
  ]
}
```

Also provided

- **Data validator script** — confirms dataset consistency and lets you self-check outputs
- **Starter repo** — data loaded, schema documented, one worked end-to-end example, stub interfaces
- **LLM API access** — keys provided at the venue (to be confirmed)

Note on the data: it targets **relational and constraint realism, not statistical realism**. Sick-call rates are not calibrated to a real carrier. What *is* guaranteed: every roster is legal or explicitly flagged, every duty clock sums correctly, every pairing obeys the ruleset. Your reasoning is therefore objectively checkable against the rules and answer keys.

5. Expected Output

A **working prototype**, demonstrated live:

1. **Conversational interface** — web chat, voice, or a well-designed CLI
2. **Reasoning layer** answering questions across as many tiers as you reach
3. **Visible explanations** on all non-trivial answers
4. **Architecture diagram** — showing the boundary you drew between LLM reasoning and deterministic logic
5. **README** — setup, approach, key trade-offs, known limitations
6. **Sample inputs and outputs**, including at least one case your system handles poorly, with your analysis
7. **Presentation deck and live demo**

Honest failure analysis scores well; overstating capability scores badly.

6. Assumptions & Constraints

Mandatory

- Use the provided synthetic dataset — no external or real airline data
- Natural language must be the primary interface
- Non-trivial answers must be explainable
- Answers must be grounded in the data — invented facts are treated as failures, not rounding errors

Optional enhancements

- Voice interface
- Multi-turn conversation with context retention
- Proactive alerting (“three crew approach duty limits tomorrow”)
- Drafting crew notifications
- Chained or simultaneous disruptions
- Confidence / uncertainty signalling

You are NOT expected to build

- **A prediction model.** Disruption-risk signals are provided pre-computed. Treat them like a weather forecast; your job is what the controller does *about* it.
- Authentication, user management or multi-tenancy
- Production infrastructure, CI/CD or deployment pipelines
- Integrations with real airline systems
- A mobile application
- A full mathematical optimisation solver — heuristic ranking with clear reasoning is sufficient

- Coverage of all real regulations — the seven provided rules are the full scope

Data limitations

- The dataset is clean; real operational data is not. Handling malformed input is a bonus, not a requirement.
- Volume is small by design — retrieval strategy is a design choice, not a scaling necessity.
- Base rates are synthetic and not statistically calibrated.

Performance

Answers should arrive fast enough to feel usable on a live shift. No formal benchmark, but a 45-second response is not a decision aid.

Security

No real personal data is involved. Commentary in your README on how such a system would handle crew PII in production earns credit under Technical Excellence.

7. Evaluation Criteria

Criterion	Weight	What we look for
AI Utilization	**20%**	Is AI solving a real reasoning problem, or decorating a lookup? How deliberately is the LLM / deterministic boundary drawn?
Innovation & Problem Solving	**15%**	Thoughtfulness of approach; non-obvious insight into the problem
Technical Excellence	**15%**	Architecture, code quality, engineering judgement, sound trade-offs
Functionality	**15%**	Does it work? How high up the tiers does it reliably reach?
User Experience	**10%**	Would a controller under pressure find this usable and trustworthy?
Presentation	**10%**	Clear articulation of problem, approach, trade-offs and limitations
Business Impact	**5%**	Credible link to operational value — time saved, disruptions avoided, cost reduced
Scalability	**5%**	Would the approach hold at real airline scale? Reasoned commentary counts
Performance	**5%**	Responsiveness and reliability

Team collaboration, role distribution and technical hurdles overcome are assessed by the jury panel per the event's standard parameters.

Scoring principles

- **A polished, reliable Tier 1** with a credible Tier 2 attempt beats a broken Tier 3
- **Correctness outweighs coverage** — answering ten questions correctly and saying “I can't answer that reliably” on the eleventh scores higher than answering all eleven with three wrong
- **Explainability is weighted throughout**, not scored in isolation

Benchmarks

Level	Description
Minimum viable	Working conversational interface answering Tier 1 accurately and clearly
Strong	Tier 2 consequence reasoning working end-to-end on a disruption scenario, with clear explanations
Exceptional	Tier 3 ranked recommendations with cost and trade-off reasoning, plus honest handling of the system's own limits

Submissions may additionally be run against a small set of **held-out scenarios** not in the starter pack, to test generalisation.

8. Deliverables Checklist

- ☐ Source code repository (GitHub / GitLab)
- ☐ Architecture diagram — including the LLM vs. deterministic boundary
- ☐ README with setup instructions, approach and trade-offs
- ☐ Sample inputs and outputs, including one failure case with analysis
- ☐ Presentation deck
- ☐ Live demo

9. Questions

Clarifications can be raised through the event's designated channel before and during the hackathon. dCortex engineers will be available on site.

The best submission will not be the one with the most features. It will be the one whose Advisor a real crew controller would want beside them at 6 a.m. on a bad day — because it is fast, because it is right, and because when it isn't sure, it says so.